



Caching Strategies

Redis · Cache-Aside · @Cacheable · Architecture Clinic #1



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Monday

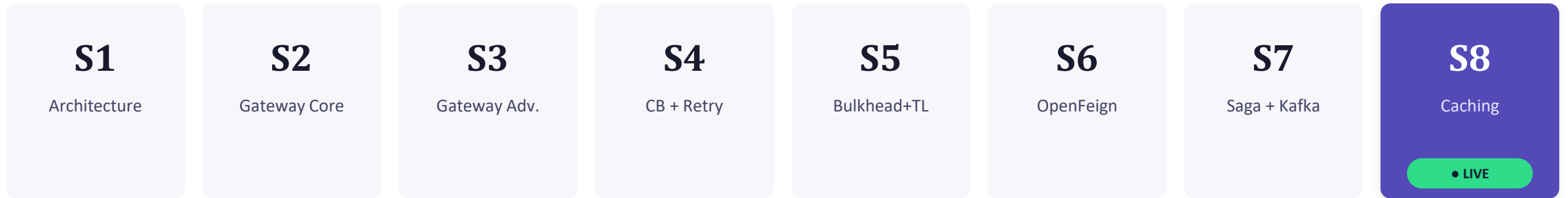
DURATION

2.5 Hours · Online

PHASE

Phase 1 — Final Session

Phase 1 — Foundation & Core Patterns



Today is the last session of Phase 1 — and it's different. We don't add a feature. We make the platform faster, then turn around and review every decision we've made since Session 1.

Full 29-session roadmap: see Course Kickoff deck



PHASE TRANSITION POINT

“Is What We Built Good Enough?”

Sessions 1–7

Every session added a feature: Gateway, security, resilience, Feign, Saga. Every session ended with something new working.

Session 8 — Today

No new service. No new annotation type.

We ask:

- *Is our system fast enough?*
- *Did we make the right architectural decisions?*

Database Reads Become a Bottleneck at Scale

WITHOUT CACHE

- GET /api/products/1 → Product Service → SELECT * FROM products WHERE id=1
- PostgreSQL responds in 50ms
- At 1000 requests/minute: 1000 queries/min hit the database directly
- PostgreSQL CPU climbs to 90% 📉
- Every single request pays the full 50ms

WITH CACHE-ASIDE (REDIS)

- GET /api/products/1 → Product Service → GET product:1 from Redis (1ms)
- Cache HIT → return immediately, no database touched
- At 1000 requests/minute: 999 hits (1ms each), 1 miss (first time only)
- PostgreSQL CPU stays under 5% 📈
- Response time: 1ms on a cache hit

When Does Caching Make Sense?

1

Is this data read much more than it's written?

Products: read 10,000x/day, updated 2x/day → GOOD candidate.
Order status changes constantly → BAD candidate.

2

Is occasional stale data acceptable?

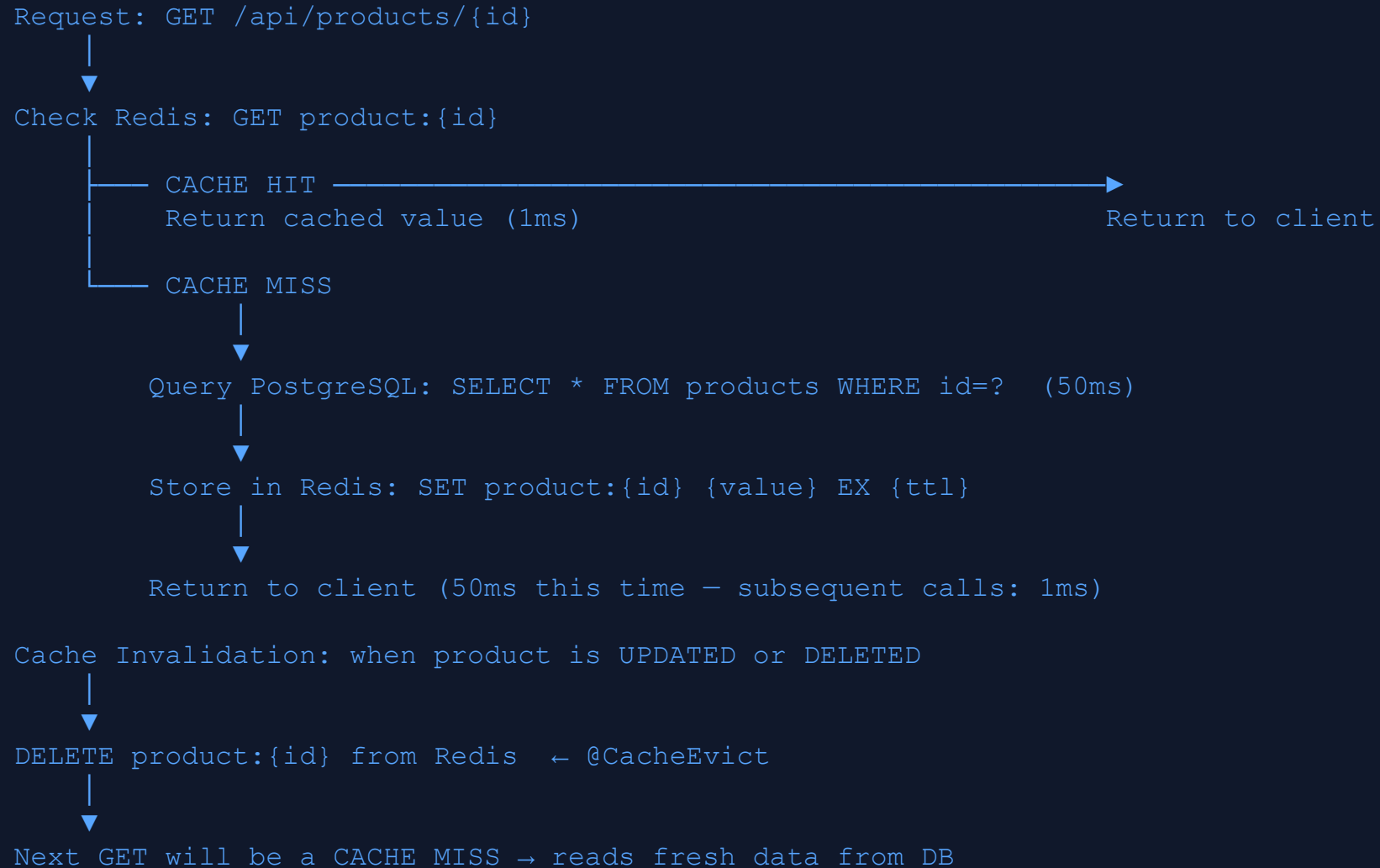
Product name/price: a short window is OK → cache with short TTL.
Account balance: must be current → do not cache.

3

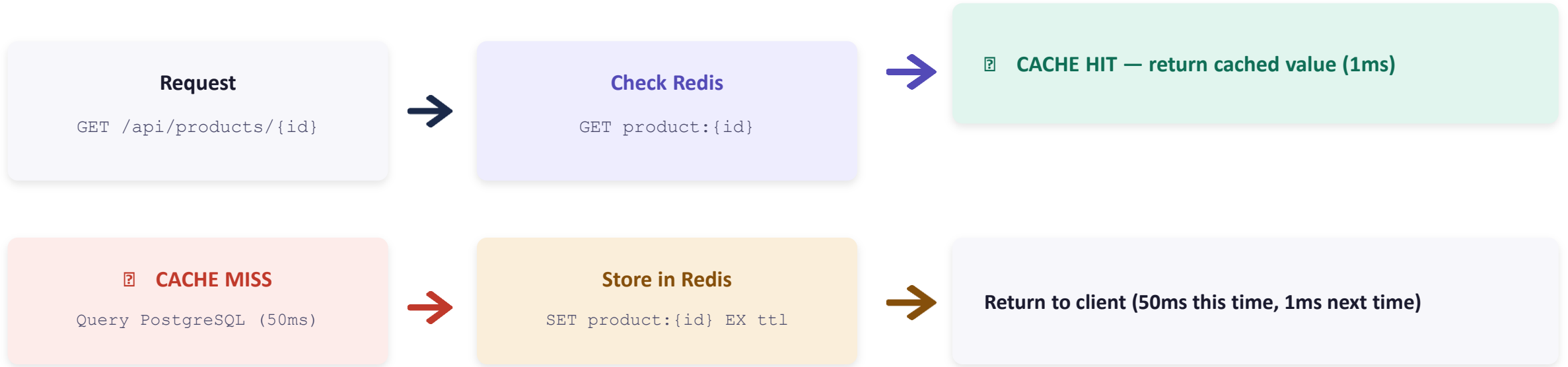
Is the cost of a cache miss acceptable?

Simple product lookup: DB query is cheap → fine either way.
Complex 10-table join report (500ms) → definitely cache.

Cache-Aside — Decision Flow: Hit, Miss, Populate



Cache-Aside — Decision Flow: Hit, Miss, Populate



Cache Invalidation — the hard part

When the product is UPDATED or DELETED → DELETE product:{id} from Redis (@CacheEvict).
Next GET is a cache miss → reads fresh data from the DB.

What to Cache — and What Never to Cache

GOOD CANDIDATES

- Product catalogue — names, descriptions, categories
- Pricing — with a short TTL (5 min)
- Configuration data — rarely changes, read constantly
- Expensive computed reports — aggregate calculations

NEVER CACHE

- Order status — changes on every Saga step
- Payment information — must be real-time accurate
- Inventory stock levels — customer might order out-of-stock
- Authentication tokens — stale tokens are a security risk

Redis Dependencies & Configuration

POM.XML — Add Redis and Spring Cache dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-data-redis
  </artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```

APPLICATION.YML — Configure Redis — PRODUCT SERVICE

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
  cache:
    type: redis
    redis:
      time-to-live: 300000 # 5 min TTL(milliseconds)
      cache-null-values: false # do not cache null responses
```



Redis is already running (Docker Compose since Session 1). No infrastructure setup needed today.

@EnableCaching on the Main Application Class

```
@SpringBootApplication
@EnableCaching // REQUIRED — activates the Spring Cache abstraction
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```



Without `@EnableCaching`, every `@Cacheable` and `@CacheEvict` annotation is silently ignored — the method always executes, Redis is never checked.

Two Annotations, Two Jobs



@Cacheable

On a READ method. Checks Redis first. On a hit, returns immediately.
On a miss, runs the method and stores the result.



@CacheEvict

On a WRITE method. Removes a key from Redis so the next read is forced to go back to the database.

- findById() and findAll() get @Cacheable — they're reads
- update() and deleteById() get @CacheEvict — they're writes that must invalidate stale data
- One more rule: evicting the single-product cache is not enough — the all-products LIST cache must be evicted too

Cache Reads, Evict on Every Write

ProductService.java — add caching annotations

```
@Service
public class ProductService {
    @Autowired private ProductRepository productRepository;
    // Cache individual product lookups. Key: "products::1", "products::2", etc.
    @Cacheable(value = "products", key = "#id")
    public Product findById(Long id) {
        log.info("[CACHE MISS] Loading product {} from database", id);
        return productRepository.findById(id).orElseThrow(() -> new ProductNotFoundException("Product not found: " + id));
    }
    // Cache all products list. NOTE: @CacheEvict on any product change must also evict "all-products"
    @Cacheable(value = "products", key = "'all'")
    public List<Product> findAll() {
        log.info("[CACHE MISS] Loading all products from database");
        return productRepository.findAll();
    }
    // Evict specific product cache AND the all-products list cache
    @CacheEvict(value = "products", key = "#product.id")
    public Product update(Product product) {
        log.info("[CACHE EVICT] Invalidating cache for product {}", product.getId());
        evictAllProductsCache(); // also evict the list cache
        return productRepository.save(product);
    }
    @CacheEvict(value = "products", key = "#id")
    public void deleteById(Long id) {
        log.info("[CACHE EVICT] Invalidating cache for product {}", id);
        evictAllProductsCache();
        productRepository.deleteById(id);
    }
    @CacheEvict(value = "products", key = "'all'")
    public void evictAllProductsCache() { /* called internally on any write operation */ }
}
```

Watch the Cache Behavior Live

```
$ curl http://localhost:8081/api/products/1      # Log: [CACHE MISS] Loading product 1 from database
$ curl http://localhost:8081/api/products/1      # Log: (nothing logged — served from Redis)

# Test 3: Verify in Redis directly
$ docker exec -it redis redis-cli
> KEYS products*
> GET "products::1"
> TTL "products::1"    # shows remaining seconds before expiry

$ curl -X PUT http://localhost:8081/api/products/1 # Log: [CACHE EVICT] Invalidating cache for product 1
$ curl http://localhost:8081/api/products/1      # Log: [CACHE MISS] Loading product 1 from database
```

The number speaks for itself: 50ms (database) vs 1ms (Redis).

- First call: MISS — hits PostgreSQL, populates Redis
- Second call: silent HIT — served entirely from Redis, no DB query, no log line
- After PUT: EVICT — the next GET is forced back to MISS, fresh data guaranteed

Cache-Aside Solves Performance — Not Every Problem



“There are only two hard things in Computer Science: cache invalidation and naming things.” — Phil Karlton

CACHE INVALIDATION STRATEGY

- @CacheEvict on — explicit eviction on write operations — this is your **PRIMARY** invalidation strategy
- TTL (Time-to-Live) — automatic expiry after N seconds — use as a **SAFETY NET** (e.g. 5–10 minutes) — even if a write path is missed, stale data eventually expires

COMBINED STRATEGY, EVERY TIME

- @CacheEvict on every write + TTL as safety net (e.g. 10 minutes)
- Even if a write path is missed, TTL ensures stale data does not persist forever.
- Never rely on TTL alone — “it’ll expire eventually” is not a consistency guarantee for anything that matters



ARCHITECTURE CLINIC #1

Be the Harshest Critic of Your Own Work

- Imagine you're a senior engineer reviewing a colleague's code for the first time — you are not here to praise it, you're here to make it better
- A lazy answer: “It works, so it's fine.”
- A good answer: “It works, but if I were to do it again, I would...”
- An excellent answer: “I see a specific risk in this design: ...”

Service Boundaries & Communication Choices

Q1

Why is Product Service separate from Inventory Service?

🔍 EXPECTED

Different team ownership, different change rate, different scaling needs.
Products rarely change; inventory changes on every order.

🚩 RED FLAG

“Because we were told to separate them” — no business justification.

Q2

Why did we use Saga instead of REST for Order → Payment?

🔍 EXPECTED

Payment is a write operation — if it fails mid-flow, we need compensation.
REST is request-reply with no built-in compensation mechanism.

🚩 RED FLAG

“Because Kafka is better than REST” — wrong comparison entirely.

Where Did We Put the Circuit Breaker — and Why There?

EXPECTED ANSWER

On the Order Service's outgoing call to Payment Service.

Order Service cannot afford to wait for a broken Payment Service — it needs to fail fast and return a degraded response instead of hanging.

RED FLAG ANSWER

“We put it everywhere to be safe” — this shows no design intent. A Circuit Breaker on every method just adds noise; harmless exceptions open circuits that didn't need to be there.

Probe deeper: “What would go wrong if we changed that?” — the learning is in the reasoning, not the answer.

Hindsight & Honest Trade-offs

Q4

What is the most rushed architectural decision we made?

🔍 EXPECTED

Open discussion.

Good answers: in-memory store for Inventory, hardcoded public routes in JWT filter, no idempotency on payment retry.

🔍 RED FLAG

“Nothing — we designed it perfectly.” No engineer, ever.

Q5

If we had to redesign one service boundary, which one?

🔍 EXPECTED

Open discussion.

Examples: should Notification be part of Order? Should Product and Inventory share a DB for simplicity?

🔍 RED FLAG

“I would not change anything.” No critical thinking applied.

Five Insights to Carry Forward

- **Service Boundaries:** confirmed the current split is justifiable on business domain lines
- **Communication:** Saga was the right choice for Order→Payment — not just because Kafka is trendy
- **Resilience Placement:** Circuit Breaker on the outgoing call is correct, not scattered everywhere
- **Most Common Debt:** in-memory stores, no idempotency, hardcoded config values
- **Key Takeaway:** architecture is not a one-time decision — it evolves with the system

Architecture Clinic #2 is Session 24 — after Kubernetes, Observability, and Security. Same list, more context.

What We're Choosing to Fix Later — On the Record

- In-memory stock map in Inventory Service — no persistence across restarts
- No idempotency on payment retry — duplicate charges possible
- PUBLIC_ROUTES list hardcoded in JwtAuthFilter — should be in config
- No dead-letter queue on Kafka consumers — failed events silently lost

- payment.failure-rate still set to 0.5 — leftover test configuration
- No structured logging — plain text log lines, hard to search at scale
- No API versioning strategy — /api/products with no version prefix
- Missing database migrations (Flyway/Liquibase) — schema deployed manually

This is not a confession — it's an engineering decision log. We revisit this exact list at Architecture Clinic #2 (Session 24).

Performance Optimization & Architecture Validation

📋 Capability Added

Performance + Architecture Validation

- Product reads now served from Redis in ~1ms.
- Architecture has been formally reviewed and technical debt documented.

📋 Services Updated

- Product Service — @Cacheable + @CacheEvict, 5 min TTL
- Platform — boundaries reviewed, tech debt logged

📋 PHASE 1 COMPLETE — Foundation & Core Patterns

8 Sessions · 20 Hours · Discovery + Gateway + Resilience + Communication + Saga + Caching, all reviewed and validated



Config+Eureka



Gateway Secured



Resilience Full



Feign Sync



Saga + Kafka



Product Cached



Arch Reviewed



Docker (S9)



DAILY QUIZ

8 Questions · 10 Minutes

Cache-Aside · TTL · @CacheEvict · Caching Candidates · Architecture Trade-offs · Technical Debt

Homework, Checkpoint & Session 9 Prep

DEFINITION OF DONE — SESSION 8

- @Cacheable on findById() — MISS→HIT logs verified
- @Cacheable on findAll() — list cached in Redis
- @CacheEvict on update() and deleteById()
- TTL: redis-cli TTL shows < 300 seconds
- 3 unit tests passing: mvn test green
- Technical debt list written in team notes

📌 📌 SESSION 9 IS A 5-HOUR OFFLINE DAY

- Have ALL services runnable locally before arriving
- Docker Desktop installed and running
- docker compose up -d works without errors
- Commit all Session 8 work before Session 9 starts

KNOWLEDGE CHECK — BEFORE SESSION 9

- What is a multi-stage Docker build and why does it produce smaller images?
- Why should production Docker images NOT run as root?

5 Real Errors, 5 Real Fixes

Issue	Solution
“No cache manager found” at startup	Missing <code>@EnableCaching</code> on <code>ProductServiceApplication</code> — this annotation activates Spring's Cache abstraction.
@CacheEvict not evicting — MISS every call	Key expression must match <code>@Cacheable</code> exactly. <code>products::1</code> vs <code>products:1</code> are different keys — use <code>key="#id"</code> .
Cache always MISS, never hits Redis	Check <code>spring.cache.type=redis</code> in yml. Without it, Spring defaults to in-memory cache, not Redis.
Cannot connect to Redis (ConnectionRefused)	Redis must be running: <code>docker compose ps grep redis</code> . Verify <code>host=localhost</code> and <code>port=6379</code> .
“Serialization failed” when caching	Product entity must implement <code>Serializable</code> , or configure a Jackson serializer for Redis.



SESSION 8 COMPLETE — PHASE 1 FOUNDATION DONE 🚩

20 Hours. 8 Sessions. One Platform.

Discovery · Gateway · Resilience · Communication · Saga · Caching — reviewed and validated

NEXT: PHASE 2 BEGINS

Session 9 — Docker + Mid-Course Exam (5h Offline Anchor Day)

WHEN

Wednesday, 10:00 AM–3:00 PM